Assignment 4

Group 7: Fina, Jenrik, David, Jesper

Heuristic Additions

- Changes to the mobility heuristic
- Modified adjacency border heuristic
- Added last_move heuristic
- Trained AI against itself

border_adjacency Heuristic

- potentially the best heuristic
- Suitable for move sorting
- preprocess an array of layers
- Set bit in layer i corresponds to max(4 - i, 0) ways to capture a stone on that tile

Task 1: Implementing a Sorting Algorithm

Objectives of sorting algorithm:

1. Reach an as ideal as possible sorting
2. Keep computational times low

Evaluation centers around mobility & border adjacency

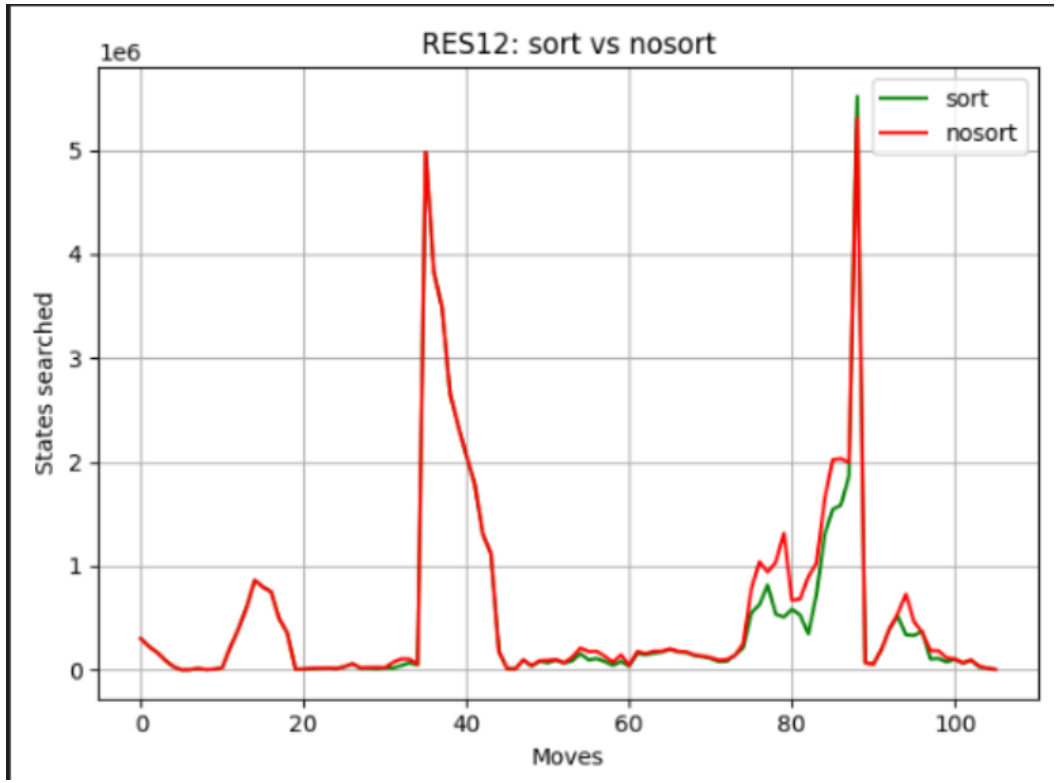
Task 1: Implementing a Sorting Algorithm

Our sorting algorithm:

1. Iterate of the adjacency layers
2. AND the adjacency layer with valid moves
3. Explore those moves

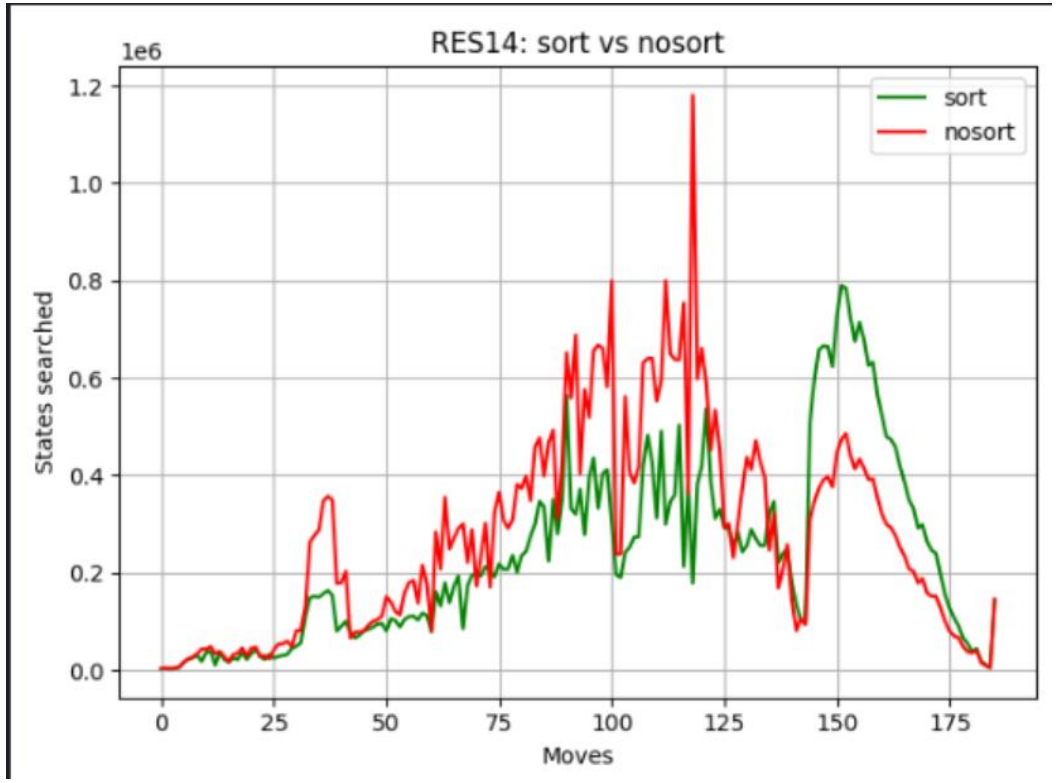
Valid corner tiles → open tiles

Task 2: Evaluating our Sorting Algorithm



- Best Case: ~ 63%
- Worst Case: 0%
- Average Case: ~ 15%
- This Map: ~ 10%

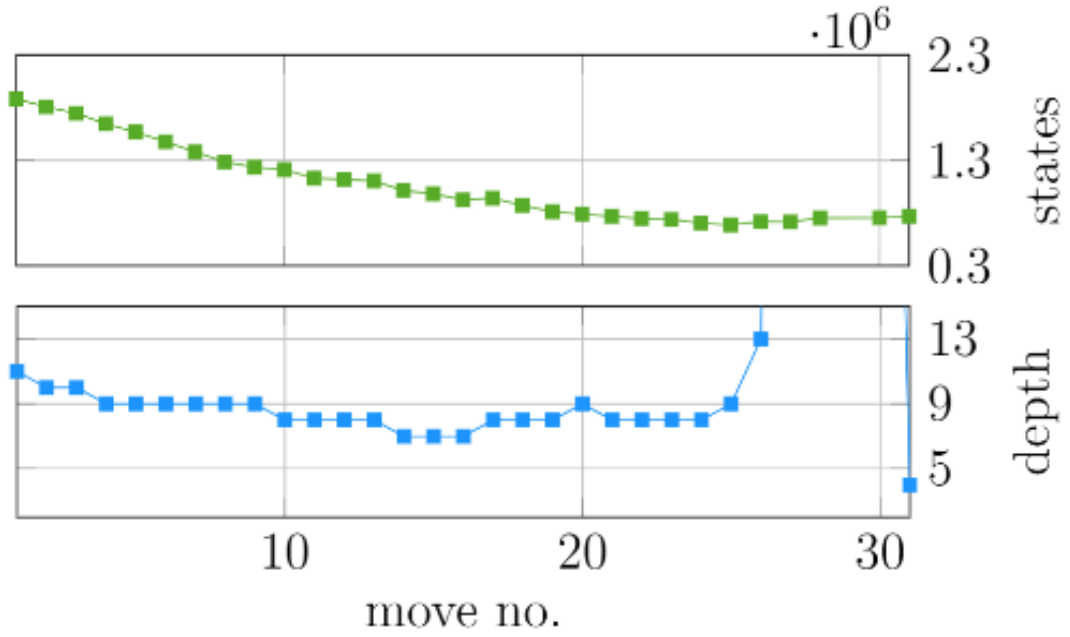
Task 2: surprising findings



Task 3: Iterative Deepening

- Global variables:
  - _time_limit
  - _abort
  - _search_start_time
- For each node searched, once the time is up, set _abort to true
- When _abort is true, all recursive functions return immediately

Task 3: Results



- 2 players, 3s time limit, 8x8
- 11 - 9 during early game
- 8 - 7 during mid game
- 100 - 200 late game

  (due to the engine being unaware that the game is finished)

Task 4: Code Cleanup

- Code Formatter: **clang-format**
- Documentation: **Doxygen**
- CI/CD: Runs tests and generates docs on every commit



https://group7-dc1ab5.pages.rwth-aachen.de/

Task 4: Code Cleanup

- Optional Build Configuration: -DBUILD_TESTING
- Links **GoogleTest** Library + GCC lib sanitizers
- Enables assertions (removes NDEBUG)

Example (Bonus)

0 0 0 0 0 0 0 0

- 1 0 0 - 1 - 0 => score 1, 3 ways to capture

0 0 0 0 0 0 0 0

0 - 0 0

0 1 – 0 => score 2, 2 ways to capture

0 0 0 0